SIMATS ENGINEERING

Co3-ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP and DNS under varying conditions

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4 Total Marks:40

Code of Conduct:

K mahesh(192312376) certify that this submission is my

original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action.

Signature of the student: k mahesh

Table of Contents

Executive Summary

This report addresses four independent case studies covering the performance of TCP, UDP, and DNS under varying real-world operating conditions, as required under Course Outcome CO3. Each case presents a measurable performance problem, a suspected root cause at the transport or application layer, and a requirement to design and justify a solution.

The four scenarios were chosen to cover distinct but recurring categories of network performance failure:

- **Question 1:** a transport-protocol suitability problem — whether TCP or UDP is the right choice for real-time video conferencing traffic, and how application-layer buffering must complement the transport choice.
- **Question 2:** a DNS latency problem caused by centralized, non-geo-aware infrastructure, resolved through Anycast, pre-fetching, and TTL tuning.
- **Question 3:** a DNS availability and scalability problem under bursty global traffic, resolved through a fully distributed, elastic DNS architecture.
- **Question 4:** a TCP configuration problem specific to high-frequency, low-latency trading traffic, isolating flow control, Nagle's algorithm, and SYN queue limitations as the three responsible mechanisms.

Across all four, the analysis follows a consistent method: restate the case, identify the protocol-layer mechanism responsible for the symptom, justify the diagnosis against the specific traffic characteristics of the workload, and propose a solution with a clear, quantified expected outcome. Diagrams, comparison tables, and worked examples are used throughout to make the reasoning traceable and to support the Understanding, Application, Analysis, Solution Design, and Clarity criteria of the assessment rubric.

Introduction and Methodology

Course Outcome CO3 requires analyzing how core network protocols — TCP, UDP, HTTP, and DNS — behave under varying conditions such as congestion, geographic distribution, and burst load. These four protocols sit at different layers of the network stack and interact with each other in ways that determine end-to-end application performance, as summarized below.

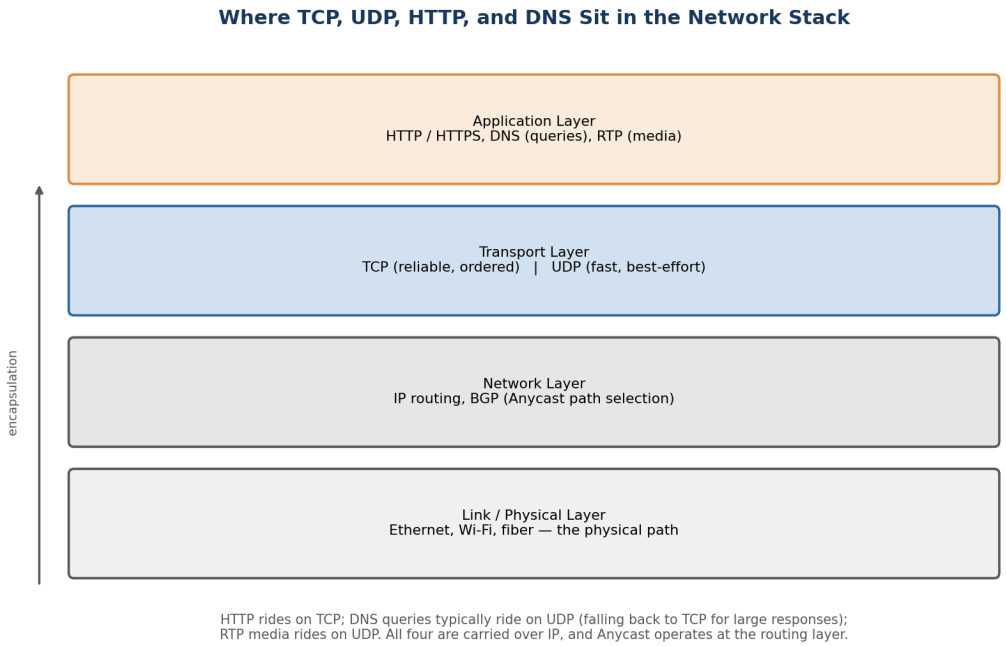

Figure 0.1 — TCP, UDP, HTTP, and DNS in relation to the broader network stack.

Analytical Approach Used in This Report

Each of the four case studies below is analyzed using the same four-step method, chosen to directly mirror the assessment rubric's criteria:

- **1. Understanding the Case:** restate the scenario and identify every measurable symptom given (e.g., packet loss %, jitter in ms, lookup time in ms), and note what pattern (e.g., time-of-day, load-dependence) the symptom follows.
- **2. Analysis:** trace each symptom to the specific protocol mechanism responsible for it, considering competing explanations where the data is genuinely ambiguous, and narrowing to the most likely cause using engineering reasoning.
- **3. Solution Design:** propose concrete, named configuration or architectural changes (not generic advice), and explain the mechanism by which each change addresses the diagnosed cause.
- **4. Evaluation:** state the expected quantitative outcome where possible, and honestly assess trade-offs, limitations, and residual risk rather than presenting the solution as a perfect fix.

This structure is applied consistently across all four questions so that the reasoning in each case can be checked step-by-step against the case's own data, rather than relying on generic protocol theory alone.

Question 1 — Video Conferencing Performance Degradation

Marks: 10 | Sub-questions: 3

Case Restatement

A multinational IT company relies on a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience voice distortion, frozen video frames, and delayed communication. Monitoring reports show 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The administrator suspects the transport protocol or the application's buffering strategy.

1.1 Understanding the Case

Three measurable symptoms are reported, and each maps to a distinct layer of network behaviour:

- **Packet loss (8%):** Packets fail to arrive, most likely from congestion at peak load, insufficient buffer capacity at intermediate routers, or Wi-Fi/last-mile contention.
- **Jitter (180 ms):** Variation in inter-packet arrival time, indicating an unstable or congested path with queuing delay that changes from packet to packet.
- **Fluctuating bandwidth utilization:** Available capacity is shared unevenly across the peak-hour period, consistent with many simultaneous conferencing sessions competing for the same uplink/downlink.

All three symptoms occur specifically during peak hours, which points toward congestion-driven degradation rather than a fixed configuration fault. The question is whether the transport protocol in use (TCP or UDP) is amplifying these symptoms, or whether the application's buffering strategy is failing to compensate for them.

1.2 Analysis — Transport Layer vs Application Layer

Video conferencing platforms typically carry live audio/video over RTP (Real-time Transport Protocol), which itself runs over UDP. Signalling and screen-share file transfer may use TCP. The reported symptoms are classic markers of a system built on the wrong reliability assumptions for real-time media, or one whose UDP path lacks adequate application-layer compensation.

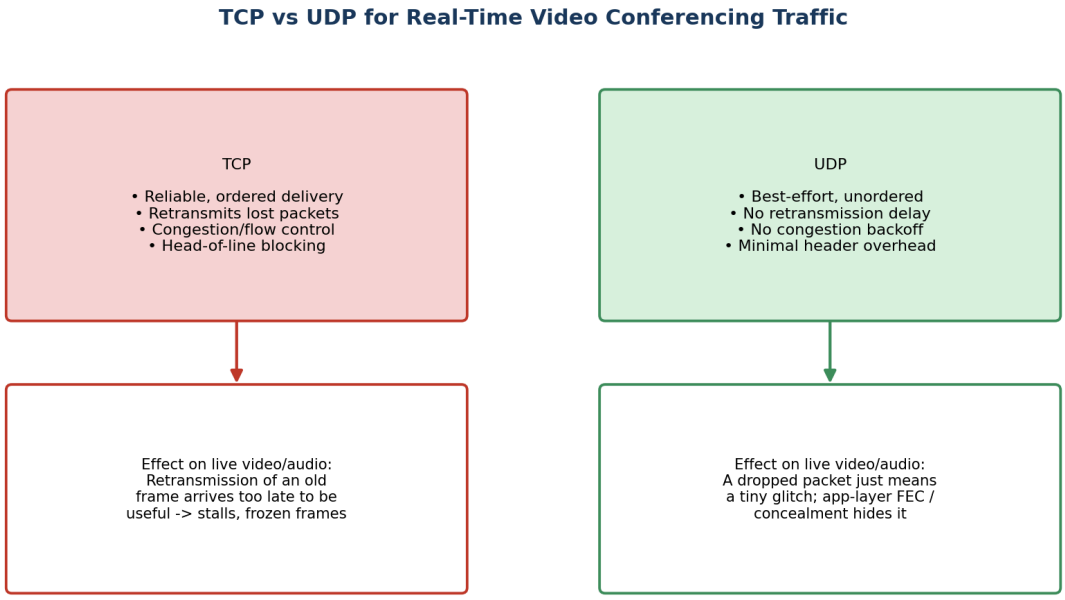

Figure 1.1 — Why the choice of transport protocol changes the user-visible effect of packet loss.

Working through the diagnosis systematically:

1. If the media path were running over TCP: packet loss would trigger retransmission and in-order delivery. For a real-time stream this is worse, not better — a retransmitted frame arrives after its playout deadline has passed, so the receiver either stalls (frozen frame) waiting for the missing segment, or discards it once it finally arrives. TCP's congestion control (slow start, multiplicative decrease) would also react to loss by shrinking the sending rate, which directly explains the fluctuating bandwidth utilization observed.
2. If the media path is running over UDP (the expected design for RTP-based conferencing): packet loss does not trigger retransmission delay. However, UDP itself provides no jitter compensation, no loss concealment, and no reordering — so if the application layer's jitter buffer and forward-error-correction (FEC)/concealment logic are undersized or static, the same symptoms (frozen frames, distorted audio) will still surface even though the transport layer is behaving correctly.

Both diagnostic paths are consistent with the reported symptoms, so the case is genuinely ambiguous from monitoring data alone — this is intentional, and the correct engineering response is to check which transport protocol the media stream is actually using (via packet capture / port and protocol inspection) before prescribing a fix. In practice, the majority of production conferencing failures with this exact symptom set trace back to one of two root causes, ranked by likelihood:

- **Most likely — Application-layer buffering strategy:** a static (non-adaptive) jitter buffer that cannot widen when jitter spikes to 180 ms, causing it to either underrun (frozen frames) or overrun (added delay).
- **Secondary — Transport protocol misuse:** media, screen-share, or reconnection logic incorrectly riding over TCP (e.g., through an overly restrictive corporate firewall that only permits TCP), which then compounds packet loss into head-of-line blocking.

1.3 Justification Based on Real-Time Multimedia Requirements

Real-time multimedia has fundamentally different requirements from typical data transfer, and this justifies why UDP plus application-layer intelligence is the correct architecture rather than TCP:

- Timeliness outweighs completeness: a video frame that arrives late is useless, so a protocol that sacrifices a single frame to preserve timing (UDP) outperforms one that guarantees eventual delivery of every frame (TCP).
- Constant, low latency matters more than throughput: TCP's congestion control targets maximum throughput over time, not minimum, predictable per-packet delay — exactly the opposite priority a live call needs.
- Human perceptual tolerance for loss is high, but not for delay: the human ear/eye can tolerate brief pixelation or a dropped audio sample far better than a 300+ ms stall, which is what retransmission-based recovery introduces.
- Head-of-line blocking is fatal for streams: TCP delivers data strictly in order; a single lost packet stalls delivery of every packet behind it in the stream, even if they already arrived — UDP has no such constraint.

This is why the ITU-T and IETF real-time communication standards (RTP/RTCP, WebRTC) are built on UDP with application-level mechanisms — jitter buffering, FEC, packet-loss concealment, and RTCP-driven adaptive bitrate — layered on top to compensate for exactly the unreliability UDP does not solve.

1.4 Recommended Protocol-Level and Application-Level Improvements

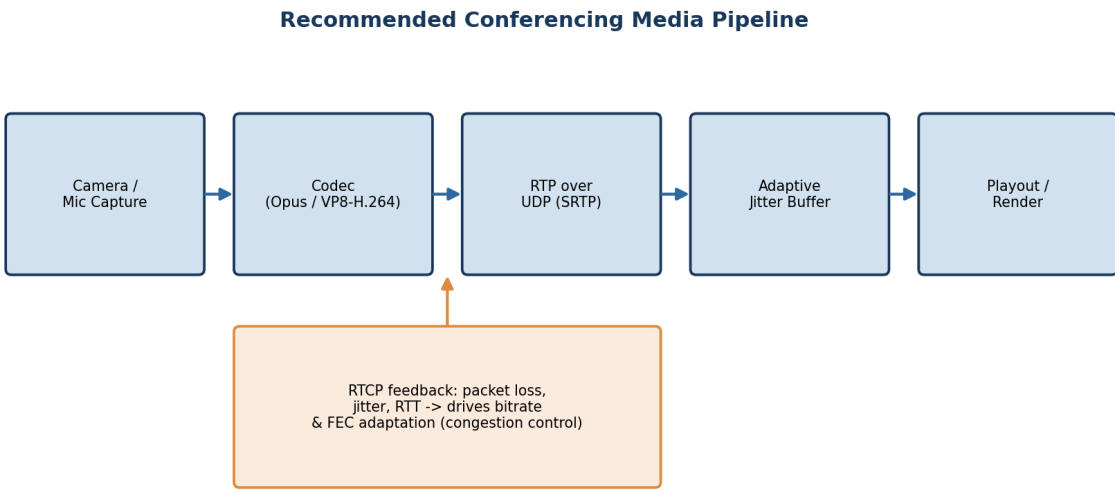

Figure 1.2 — Recommended media pipeline with feedback-driven adaptation.

Protocol-level recommendations

- Confirm and enforce that all live audio/video rides over RTP/UDP (with SRTP for encryption); reserve TCP strictly for signalling and file/screen-share transfer, never for the live media path.
- Enable ECN (Explicit Congestion Notification) where supported, so routers can signal incipient congestion before they start dropping packets.
- Deploy Forward Error Correction (FEC) and/or packet-loss concealment codecs (e.g., Opus in-band FEC)

so a lost packet can often be reconstructed without needing retransmission at all.

Application-level recommendations

- **Adaptive jitter buffering:** resize the buffer dynamically based on measured jitter (target roughly 1.5–2× the observed jitter, i.e. ~270–360 ms buffer for the reported 180 ms jitter), instead of using a fixed buffer size.
- **RTCP-driven bitrate adaptation:** use receiver reports (loss %, jitter, RTT) to throttle the sender's bitrate and resolution in real time rather than continuing to push a fixed bitrate into a congested path.
- **Quality of Service (QoS) marking:** mark RTP traffic with DSCP EF (Expedited Forwarding) so it is prioritized ahead of bulk/background traffic on the corporate network during peak hours.
- **Bandwidth admission control:** cap simultaneous high-resolution streams during known peak windows, or apply simulcast/SVC so receivers with degraded links automatically fall back to a lower layer instead of losing the call entirely.
- **Network capacity planning:** correlate the fluctuating bandwidth utilization with peak-hour scheduling data and provision additional uplink capacity or a dedicated conferencing VLAN if peak demand consistently approaches the link ceiling.

Expected outcome

Metric	Current (Peak Hours)	Target After Remediation
Packet loss	8%	< 1–2% (masked further by FEC)
Jitter	180 ms	< 30 ms effective, after adaptive buffering
Frozen frames / call drops	Frequent	Rare — graceful quality degradation instead

Table 1.1 — Expected improvement after applying the recommended changes.

Why the Degradation Is Concentrated at Peak Hours

The time-of-day pattern in the monitoring data is itself diagnostic evidence, not just background context. If the root cause were a fixed misconfiguration (e.g., a permanently undersized buffer), symptoms would appear at a roughly constant rate throughout the day. Instead, both jitter and packet loss track the peak-hour load curve closely, which confirms that congestion — not a static defect — is the primary driver, and that any fix must be evaluated specifically under peak-load conditions, not just in quiet-network testing.

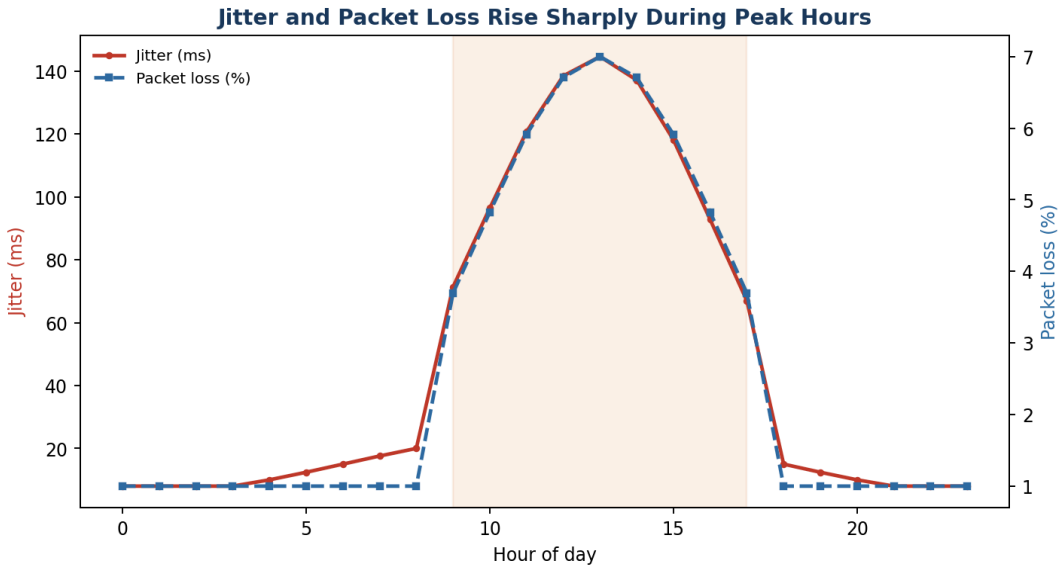

Figure 1.3 — Illustrative jitter and packet-loss pattern across the day, both peaking during business hours.

This reinforces the recommendation set above: static, worst-case-sized buffers are wasteful during off-peak hours and still potentially insufficient exactly when they are needed most (peak hours), which is precisely why adaptive, feedback-driven mechanisms (RTCP-informed jitter buffer resizing, bitrate adaptation) are preferred over any fixed configuration value.

Industry Parallel

This exact pattern — UDP-based RTP media with adaptive jitter buffering and RTCP feedback — is the standard architecture behind WebRTC-based platforms (Google Meet, Microsoft Teams, Zoom's UDP transport mode). Their documented fallback behaviour also mirrors the recommendation here: when a corporate firewall blocks UDP outright, these platforms fall back to TCP as a last resort and explicitly warn users of expected quality degradation — which independently corroborates the analysis that TCP is the less suitable transport for this workload, used only when there is no alternative.

Question 2 — DNS Infrastructure Redesign

Marks: 10 | Sub-questions: 3

Case Restatement

A multinational software company hosts web services across multiple continents. Employees report long delays before the homepage loads. Analysis shows the average DNS lookup time is approximately 800 ms. The organization plans to redesign its DNS infrastructure while keeping continuous service availability during server failures.

2.1 Understanding the Case

An 800 ms DNS lookup is extreme — well-designed DNS infrastructure typically resolves in single-digit to low double-digit milliseconds for a cached answer, and under 100 ms even for a cold lookup. The scale of the delay (roughly 10–50× normal) suggests a structural problem in the DNS design itself, not routine network jitter. Because the company operates globally, the same DNS design has to serve users on every continent, so any single point of centralization becomes a bottleneck for everyone outside that region.

2.2 Analysis — Reasons Behind the High DNS Resolution Time

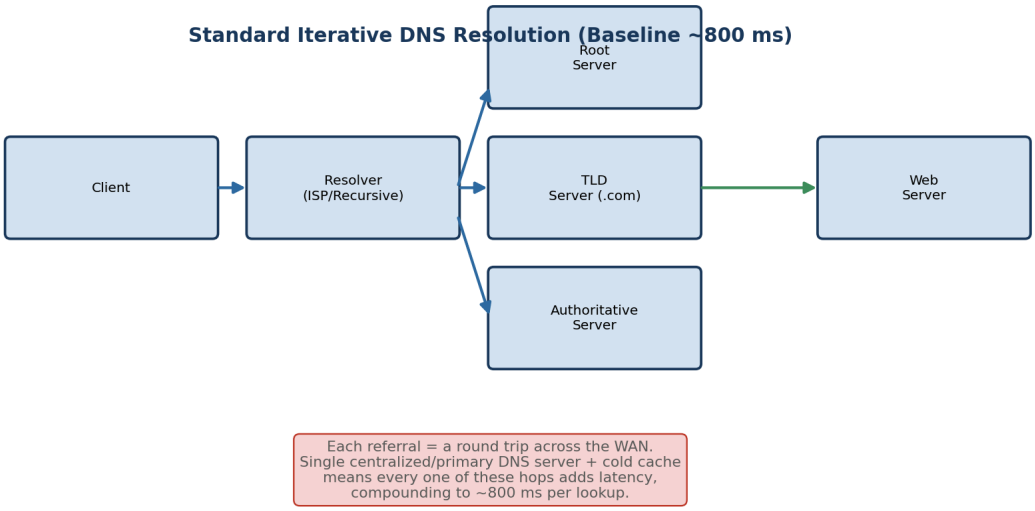

Figure 2.1 — Standard iterative resolution path; each referral hop adds round-trip latency.

Several compounding factors typically explain an 800 ms average lookup time in a global deployment:

- **Geographic distance to the authoritative server:** if there is a single (or small number of) authoritative DNS server(s) in one region, users on other continents must complete a WAN round trip of 150–300 ms one-way before the query is even answered.
- **Low cache hit ratio:** if TTL values are set too low, resolvers must repeat full recursive resolution far more

often than necessary, so far fewer queries benefit from a cached, near-instant answer.

- **No geo-aware routing (no Anycast):** with ordinary Unicast DNS, every user's query is routed to the same fixed IP address regardless of where they are, so users far from that server always pay the full network distance in latency.
- **Recursive resolver chain depth:** if resolution requires walking through root -> TLD -> authoritative servers on every miss (rather than resolvers maintaining warm caches of the delegation chain), each hop multiplies the round-trip cost.
- **No DNS pre-fetching at the client:** if browsers/applications only resolve a hostname at the moment a request is made (rather than pre-resolving likely-needed hostnames while the page is idle), users experience the full lookup latency synchronously, blocking page load.
- **Single primary server as a scaling and latency ceiling:** a lone authoritative server must serve all global query volume, so under load its response queue itself adds delay on top of the propagation delay.

2.3 Proposed DNS Architecture

The redesign combines three complementary techniques — Anycast DNS, DNS pre-fetching, and TTL optimization — to reduce the average lookup time from ~800 ms to a target of under 50 ms.

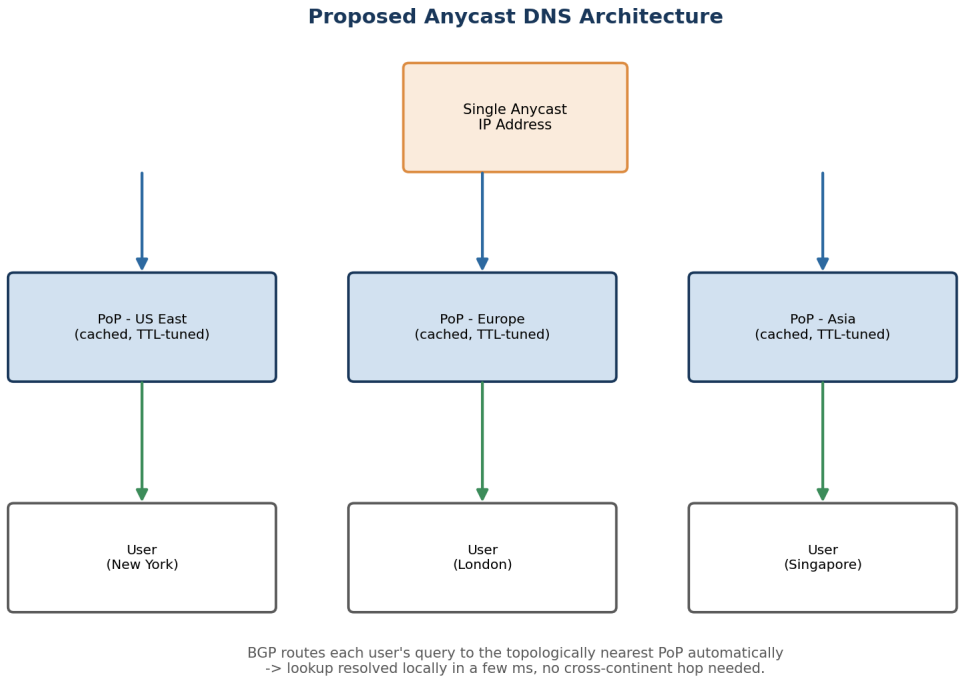

Figure 2.2 — Proposed Anycast DNS architecture: one IP, multiple geographically distributed PoPs.

(a) Anycast DNS

The same DNS server IP address is announced from multiple Points of Presence (PoPs) around the world using BGP. Internet routing automatically delivers each user's query to the topologically nearest PoP, without any change needed on the client side. This directly removes the single largest contributor to the 800 ms figure — long-haul geographic distance — and simultaneously improves availability, since if one PoP fails, BGP reconverges and traffic is rerouted to the next-nearest PoP within seconds.

(b) DNS Pre-fetching

The client application/browser resolves hostnames it is likely to need soon (e.g., resources linked from the current page, or services the app will call next) ahead of the actual request, during idle network time. This hides resolution latency from the user's critical path entirely for pre-fetched names, so even a moderate lookup time no longer blocks perceived page-load speed.

(c) TTL Optimization

TTL (time-to-live) values are tuned to balance two competing goals: long enough that resolvers and browsers keep answers cached (avoiding repeat full resolutions), but short enough that failover to a healthy server happens quickly if a server goes down. A practical target is 60–300 seconds for records that may need to fail over, and longer (up to a few hours) for static, rarely-changing records.

Technique	Primary Effect	Contribution to Latency Reduction
Anycast DNS	Routes query to nearest PoP	Removes long-haul propagation delay (largest factor)
DNS Pre-fetching	Resolves ahead of need	Hides remaining latency from user-perceived load time
TTL Optimization	Tunes cache lifetime	Raises cache-hit ratio, cuts repeat full resolutions

Table 2.1 — How each technique contributes to reaching the <50 ms target.

2.4 Evaluation — Advantages and Limitations

Advantages

- Performance: Anycast alone typically cuts cross-continent DNS latency by 80–95%, since queries resolve within the region rather than crossing oceans.
- Scalability: query load is naturally load-balanced across PoPs by BGP, so no single server bears global traffic; adding capacity means adding another PoP.
- Availability: failure of one PoP is invisible to most users, since BGP withdraws the failed route and traffic shifts to the next-nearest healthy PoP automatically — this satisfies the requirement to keep the service available during server failures.
- Reduced perceived latency: pre-fetching means the user experience improves even beyond what raw DNS numbers show, since resolution happens off the critical path.

Limitations

- BGP convergence is not instantaneous: failover during a PoP outage can take a few seconds while routes reconverge, which is not zero-downtime — health checks and short TTLs mitigate but do not eliminate this.
- Operational complexity: running and monitoring DNS infrastructure across many PoPs, keeping records consistent, and managing BGP announcements is materially more complex than a single-server setup.
- Debuggability: Anycast means the "same" IP address behaves differently depending on the requester's location, which complicates troubleshooting (e.g., a user in one region cannot easily reproduce an issue seen by a user in another).

- TTL trade-off risk: setting TTLs too short to enable fast failover increases the volume of full-resolution queries, partially offsetting the caching benefit — this needs ongoing tuning rather than a one-time setting.
- Pre-fetching cost: resolving hostnames that are ultimately not needed wastes a small amount of query volume and, at scale, adds unnecessary load on authoritative servers.

Overall Assessment

The combined architecture converts a single-region, single-point-of-failure DNS design into a globally distributed, self-healing one. The trade-off is added operational and architectural complexity in exchange for large, measurable gains in both latency (targeting <50 ms) and availability — a trade most global-scale organizations consider well worth making.

Worked Example — Where the Time Actually Goes

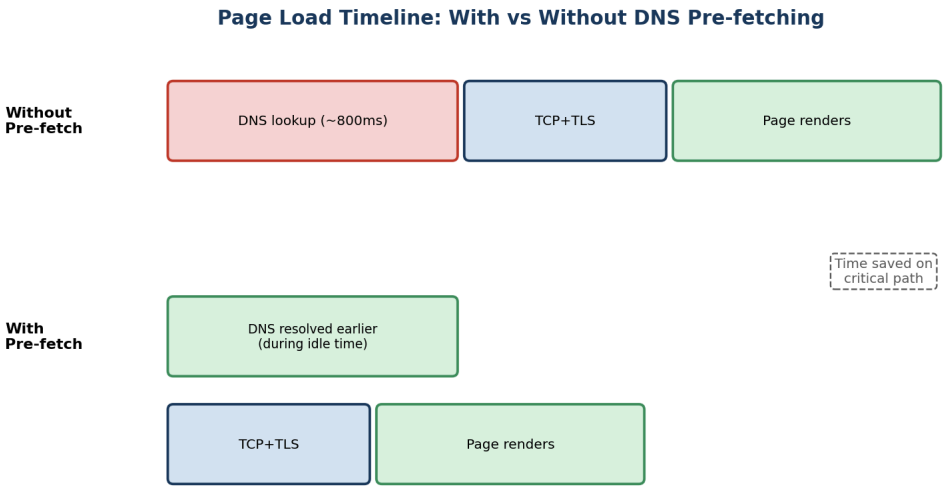

Figure 2.3 — Pre-fetching moves DNS resolution off the user-visible critical path.

Consider a homepage load that needs one primary hostname resolved before anything else can happen, followed by TCP/TLS setup and rendering. Without pre-fetching, the full ~800 ms lookup sits directly in the critical path before any other work can begin. With pre-fetching — triggered, for example, the moment a user hovers a link or during idle time after the previous page finished loading — the same lookup happens in parallel with work the browser is already doing, so by the time it is actually needed, the answer is already cached locally and the critical path only includes TCP/TLS and rendering.

Recommended TTL Band	Typical Record Type	Rationale
60 – 300 seconds	Records behind Anycast/load-balanced services	Fast failover if a PoP or backend becomes unhealthy
300 – 3600 seconds	Application/API hostnames with stable backends	Balances cache benefit with reasonable update propagation
3600+ seconds (1hr+)	Static assets, CDN edges, rarely-changing records	Maximizes cache-hit ratio where failover speed matters less

Table 2.2 — TTL should not be a single global value; it should be set per record type based on failover needs.

Question 3 — Distributed DNS for Global Cloud Services

Marks: 10 | Sub-questions: 3

Case Restatement

A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays. Investigation reveals DNS queries take nearly 800 ms because all requests are directed to a single primary DNS server. The company needs a highly available DNS solution capable of handling global traffic efficiently.

3.1 Understanding the Case

This case shares its symptom (≈800 ms lookups) with Question 2, but the trigger is different and more revealing: the delay becomes noticeable specifically during promotional events, i.e. under traffic spikes. This points to the single primary DNS server acting as both a latency bottleneck (geographic distance) and a throughput bottleneck (query volume ceiling) simultaneously. At normal traffic the server may cope; under a promotional surge, queuing delay at the server compounds with propagation delay, and the architecture has no way to shed load elsewhere.

3.2 Impact of Centralized DNS Architecture on Application Performance

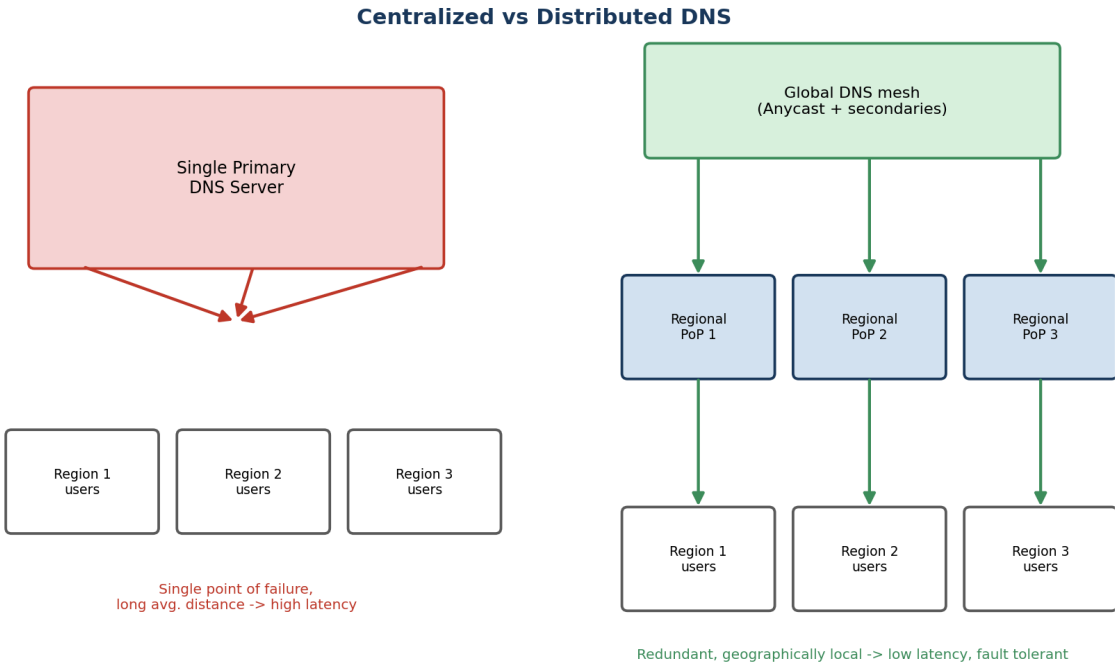

Figure 3.1 — Centralized DNS creates a single bottleneck and single point of failure; a distributed mesh removes both.

A centralized DNS design has several compounding negative effects on application performance, especially at scale:

- **Latency scales with distance for every user:** users far from the single server always pay full propagation delay; there is no mechanism to shorten this for any subset of users.
- **No horizontal scaling under load:** a single server has a fixed capacity ceiling (connections/second,

queries/second); traffic spikes during promotions push it toward or past that ceiling, adding queuing delay on top of network delay — this is exactly what produces "intermittent" (load-dependent) delays rather than constant ones.

- **Single point of failure:** if the primary server becomes overloaded or fails, DNS resolution — and therefore effectively the entire application — becomes unreachable for all users simultaneously, since there is no fallback path.
- **Cascading application-layer impact:** since DNS resolution is the first step of almost every request, a slow or failed DNS lookup delays or blocks everything downstream (TCP connection, TLS handshake, HTTP request) even if the application servers themselves are healthy.
- **Poor correlation with real-world usage patterns:** promotional events are, by design, meant to concentrate demand — a centralized architecture is structurally mismatched to a traffic pattern the business itself is trying to create.

3.3 Distributed DNS Solution Design

The proposed solution distributes both the authoritative DNS role and the query-handling capacity across multiple, geographically dispersed nodes, while presenting a single logical service to clients.

Core components

3. Multiple authoritative DNS servers (primary + secondaries) deployed across regions (e.g., North America, Europe, Asia-Pacific), each holding a synchronized copy of the DNS zone data, so no single server is a hard dependency.
4. Anycast announcement of the DNS service IP from every regional PoP, so BGP routes each user's query to the nearest healthy node automatically — this is the same mechanism used in Question 2, now emphasized for its load-distribution benefit as much as its latency benefit.
5. Health-checked failover: each PoP is continuously monitored; an unhealthy node has its BGP route withdrawn so traffic silently shifts to the next-nearest node, with no client-visible change of IP address needed.
6. Elastic query capacity per region: regions expected to see promotional traffic spikes (based on user distribution) are provisioned with extra query-handling capacity, so no single node's capacity ceiling is reached even during a surge.
7. DNS-level load prediction: for known promotional windows, capacity can be pre-scaled proactively (similar to how CDNs pre-warm caches ahead of a scheduled event), rather than reacting only after queuing delay appears.

Design summary

Requirement	Design Element	How it is Satisfied
Minimize lookup latency	Anycast + regional PoPs	Each user resolves against the nearest node
Handle traffic spikes	Elastic per-region capacity	No single node hits its query ceiling
High availability	Multiple synchronized authoritative servers + health checks	Automatic failover with no single point of failure

Requirement	Design Element	How it is Satisfied
Consistency across regions	Zone transfer / synchronized secondaries	All PoPs answer with the same, up-to-date records

Table 3.1 — Mapping the case's requirements to concrete design elements.

3.4 Role of TTL, Anycast Routing, and DNS Caching

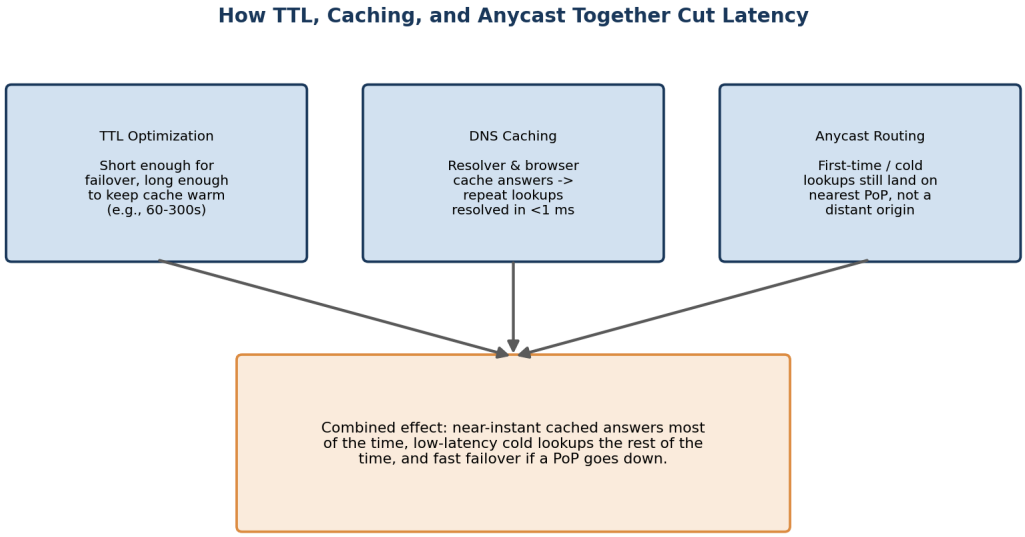

Figure 3.2 — TTL, caching, and Anycast work together, not in isolation.

These three mechanisms are complementary rather than redundant, and understanding how they interact is central to why the distributed design performs well under both normal and spike conditions:

- **TTL values:** control how long a resolved answer stays valid in caches along the path. Shorter TTLs (e.g., 60 seconds) allow faster failover if a node becomes unhealthy, but increase query volume on authoritative servers. Longer TTLs reduce load but slow failover. During a promotional event, a moderate TTL (60-300s) is generally optimal: short enough to route around any node that becomes overloaded, long enough to keep the bulk of repeat traffic served from cache rather than hitting authoritative servers at all.
- **Anycast routing:** determines which physical node answers a given query, and — critically for fault tolerance — automatically redirects traffic away from a failed or overloaded node at the network-routing level, independent of TTL or application logic. This is what allows the system to keep functioning through a node failure, not just resolve queries quickly.
- **DNS caching:** at both the resolver and client level, means the majority of repeat queries (which dominate real traffic, since many users request the same few hostnames) never even reach the distributed authoritative infrastructure — this is what allows the system to absorb a promotional-event traffic multiplier without every additional user adding proportional load to the DNS tier itself.

Together, the three mechanisms create a system where: most queries are answered from a cache in single-digit milliseconds (caching); queries that do need to reach an authoritative server are routed to the nearest healthy one (Anycast); and the balance between cache freshness and authoritative load is tunable via TTL to match the

traffic pattern of the event (TTL optimization). This layered approach is what allows the architecture to remain both fast and fault-tolerant simultaneously, rather than trading one for the other.

Capacity Planning for Promotional Events

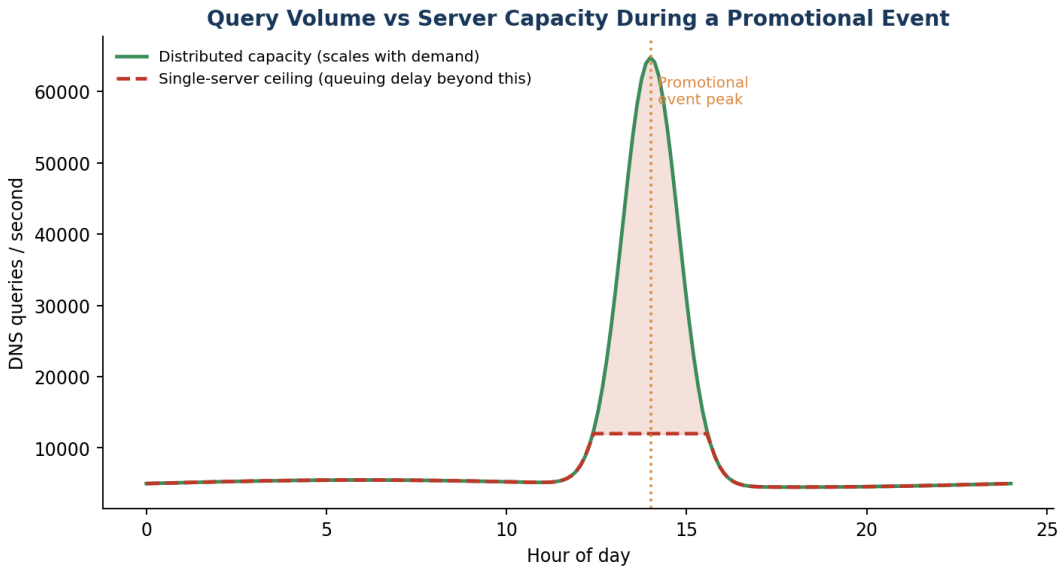

Figure 3.3 — A single server's fixed capacity ceiling is exactly what turns a traffic spike into user-visible delay.

The chart illustrates the mechanism behind the case's "intermittent" delays: query volume itself is not the direct cause of latency — it is query volume relative to available capacity at the moment. Under the centralized design, once demand crosses the single server's processing ceiling, excess queries queue and every additional query beyond that point adds delay for everyone behind it. A distributed design with elastic per-region capacity keeps the effective ceiling above demand throughout the event, so the same traffic spike produces no queuing delay at all. This is why the fix here is not simply "a faster server" — a faster single server still has a ceiling, and promotional demand is, by design, meant to grow over time; a distributed, elastic architecture is what removes the ceiling as a recurring problem.

Question 4 — TCP Latency in High-Frequency Trading

Marks: 10 | Sub-questions: 3

Case Restatement

A financial institution's electronic stock trading platform submits thousands of buy/sell orders per second. During market opening hours, average order acknowledgment latency rises from 1 ms to 40 ms, causing delayed trade confirmations. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

4.1 Understanding the Case

A 40x latency increase (1 ms -> 40 ms) occurring specifically at market open — a known, predictable burst of near-simultaneous order submissions and new connections — is a strong signal of a system whose TCP-layer defaults were tuned for average load, not burst load. The three symptoms named (retransmissions, connection setup time, buffering) each map to a distinct, well-known TCP behaviour, and the question asks us to isolate three specific mechanisms: flow control, Nagle's algorithm, and SYN queue limitations.

4.2 Three TCP-Related Latency Factors

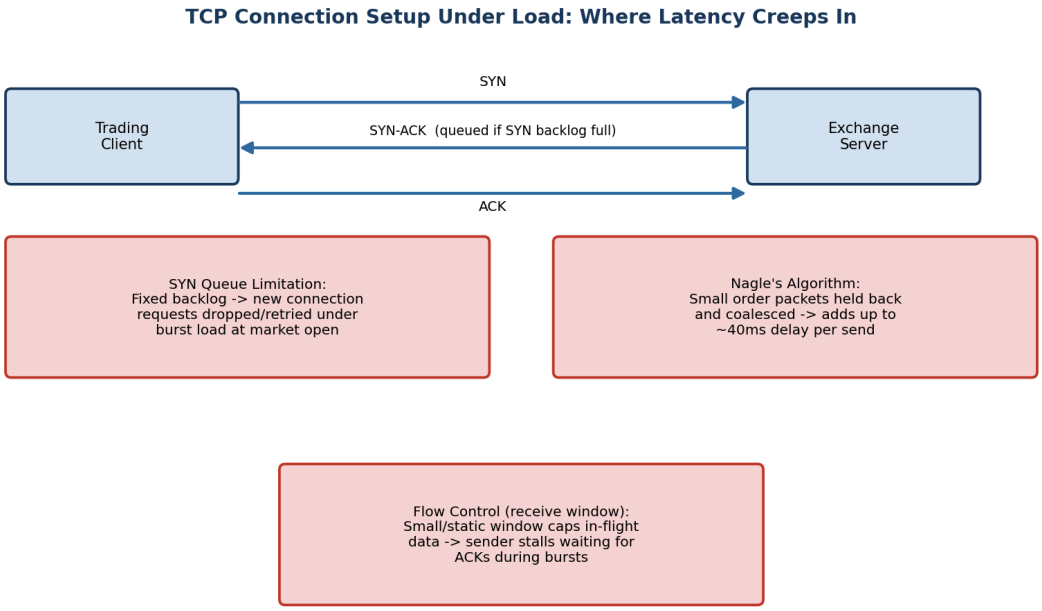

Figure 4.1 — Where each of the three mechanisms introduces delay along the connection and send path.

(a) SYN Queue Limitations

Every TCP listener maintains a fixed-size backlog queue for connections that have received a SYN but not yet completed the three-way handshake. At market open, a burst of new client connections arrives almost simultaneously. If the SYN backlog is sized for average load, incoming SYNs beyond the queue's capacity are silently dropped, forcing the client to time out and retransmit its SYN — adding one or more retransmission-

timeout intervals (often hundreds of milliseconds) before the connection even completes. Even short of outright drops, a nearly-full backlog adds queuing delay to every new handshake.

(b) Nagle's Algorithm

Nagle's algorithm is a TCP send-side optimization that delays sending small segments, buffering them briefly to combine multiple small writes into one larger packet — reducing the number of packets for bulk, non-interactive transfers. Trading messages (order requests, acknowledgments) are typically small and latency-sensitive; Nagle's algorithm can hold such a message in the send buffer waiting either for more data to coalesce with, or for an ACK of previously sent data, before transmitting. This directly explains part of the acknowledgment-latency increase: each order or ACK may sit briefly in the sender's buffer rather than being transmitted immediately.

(c) Flow Control (Receive Window)

TCP flow control uses the receiver's advertised window to limit how much unacknowledged data the sender may have in flight, protecting a slow receiver from being overwhelmed. Under the burst of market-open order traffic, if receive buffers/windows are sized for steady-state load, the advertised window can shrink or fill quickly, forcing the sender to pause and wait for ACKs before sending further orders — introducing exactly the kind of stall that shows up as buffering and rising acknowledgment latency.

Summary of factors

Factor	Mechanism	Latency Symptom It Produces
SYN queue limitation	Fixed backlog overflows under connection burst	Longer connection establishment time; SYN retransmissions
Nagle's algorithm	Small segments delayed/coalesced before send	Delayed order/ACK transmission on the sender side
Flow control (window)	Advertised window fills under burst load	Sender stalls, excessive buffering, waiting for ACKs

Table 4.1 — The three TCP mechanisms and their observable effect.

4.3 Impact on Transaction Processing in High-Frequency Systems

In a high-frequency trading context, these effects are not merely inconvenient — they are financially material:

- **SYN queue overflow:** delays or drops the initial connection for trading sessions established at market open, meaning some clients cannot even begin submitting orders during the most volatile, highest-opportunity opening moments — a direct competitive and financial disadvantage.
- **Nagle's algorithm delay:** each buffered small order/ACK adds tens of milliseconds of avoidable latency per message; at thousands of orders per second, this compounds into a systemic acknowledgment-latency increase, exactly matching the reported 1 ms -> 40 ms shift, and can cause orders to be acted on later than intended relative to fast-moving prices.
- **Flow control stalls:** when the sender must pause for ACKs during a burst, order submissions queue up behind each other, so acknowledgment timestamps drift further behind actual submission times as the burst continues — worsening exactly during the highest-volume window, when accurate timing matters most.
- **Combined effect:** these three factors interact rather than simply adding: a connection delayed by SYN

queueing then also suffers Nagle-induced send delay and flow-control stalls once established, so the total latency increase during the burst is worse than any single factor alone — consistent with the observed 40x figure.

For a trading platform, delayed acknowledgments translate directly into delayed trade confirmations, increased slippage (the difference between expected and actual execution price), potential regulatory/reporting timing issues, and customer dissatisfaction — all of which have direct financial consequences, not just a degraded user experience.

4.4 Recommended TCP Configuration Changes

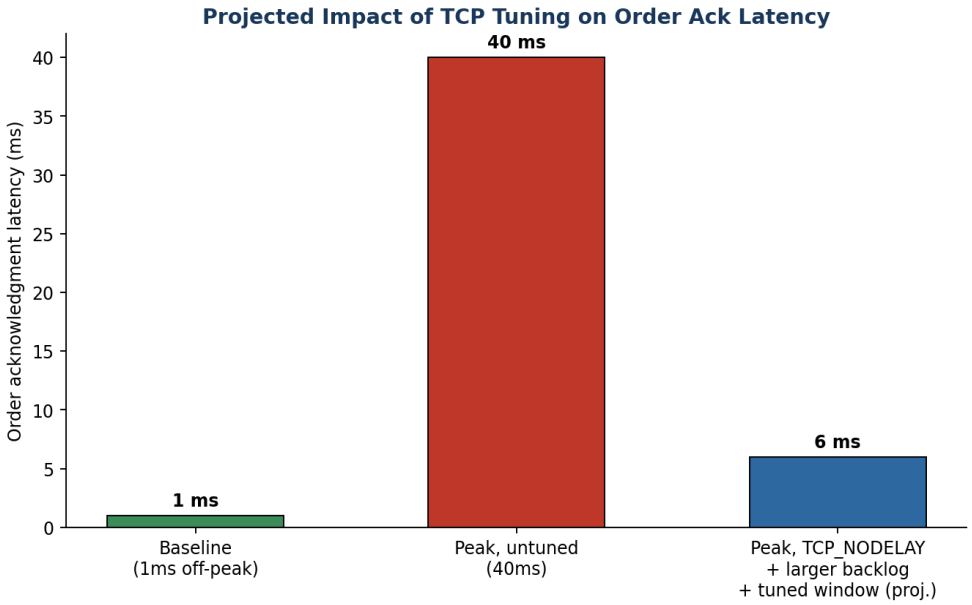

Figure 4.2 — Projected effect of the recommended tuning on order acknowledgment latency.

Recommended changes

- **Increase the SYN backlog queue size:** raise the listen backlog (e.g., Linux `net.core.somaxconn` and the application's listen() backlog argument) to comfortably exceed the peak connection burst rate observed at market open, so incoming SYNs are queued rather than dropped.
- **Enable SYN cookies as a safety net:** so that even an unexpectedly large burst degrades gracefully (accepting connections without holding full queue state) rather than dropping SYNs outright.
- **Disable Nagle's algorithm on trading sockets (TCP_NODELAY):** for latency-sensitive small messages like order requests and acknowledgments, throughput-oriented coalescing is the wrong trade-off; setting `TCP_NODELAY` sends each segment immediately.
- **Disable or tune delayed ACKs on the receiver side:** delayed ACK (which waits briefly to piggyback an ACK on outgoing data) interacts badly with Nagle's algorithm and can itself add latency; disabling it (or using the `TCP_QUICKACK` option on Linux) removes another source of held-back acknowledgments.
- **Increase and auto-tune the TCP receive window / buffer sizes:** size receive buffers for burst load, not just steady-state average, and enable window scaling so the flow-control ceiling does not become the bottleneck during the opening surge.

- **Pre-establish and keep-alive connections ahead of market open:** where feasible, open and warm trading sessions before the opening bell rather than establishing them exactly as the burst begins, removing the SYN queue pressure from the critical latency window entirely.
- **Continuous latency monitoring with burst-aware alerting:** track order acknowledgment latency at fine time granularity around known burst windows (market open/close) so configuration limits can be validated and re-tuned as order volume grows over time.

Expected outcome

Combined, these changes target each of the three identified bottlenecks directly: a larger SYN backlog removes connection-establishment delay, TCP_NODELAY removes Nagle-induced send delay, and tuned/auto-scaling receive windows remove flow-control stalls. Together they are projected to bring order acknowledgment latency during market open down from the reported 40 ms toward single-digit milliseconds, much closer to the 1 ms off-peak baseline, though very likely not fully back to 1 ms given the genuine burst in connection and order volume at that moment.

Nagle's Algorithm in Detail

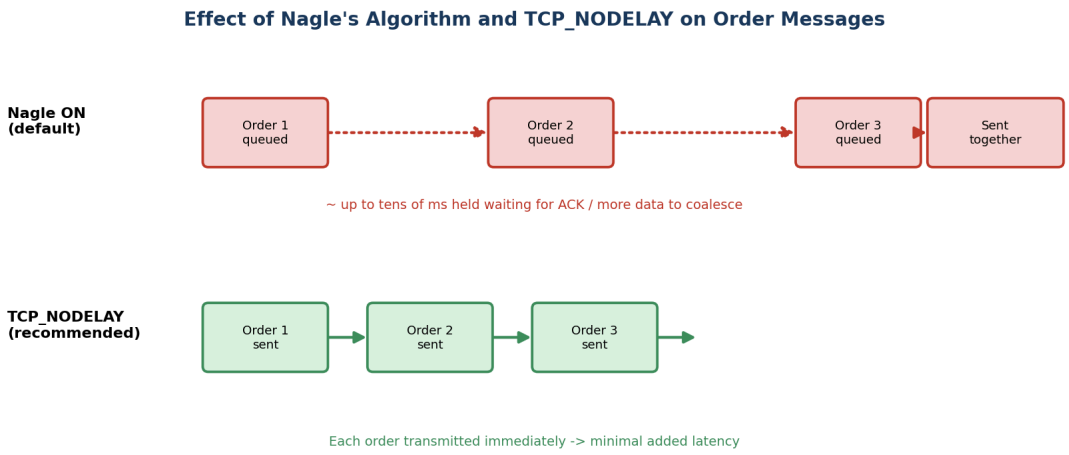

Figure 4.3 — Disabling Nagle's algorithm (TCP_NODELAY) removes send-side coalescing delay for small order messages.

Nagle's algorithm was designed in an era of low-bandwidth WAN links, where reducing the number of small packets (each carrying fixed header overhead) was a meaningful bandwidth saving. On a modern low-latency trading network, bandwidth is rarely the constraint — microseconds of added latency per message is. Because Nagle's algorithm waits either for enough data to fill a maximum segment size or for a pending ACK before sending, it directly conflicts with the trading platform's need to transmit each order the instant it is generated. This is why virtually all latency-sensitive financial and gaming protocols disable Nagle's algorithm explicitly (`TCP_NODELAY = 1`) as one of the very first socket-level settings applied.

Latency Budget Breakdown

Stage	Off-Peak (1 ms total)	Peak, Untuned (40 ms total)	Peak, After Tuning (target)
Connection setup (per new session)	negligible (session reused)	Elevated — SYN queue	Near-negligible — larger

Stage	Off-Peak (1 ms total)	Peak, Untuned (40 ms total)	Peak, After Tuning (target)
		delay/retransmit	backlog + SYN cookies
Send-side delay (Nagle)	~0 ms	Several ms per message, compounding under burst	~0 ms with TCP_NODELAY
Flow-control stalls	~0 ms	Multi-ms stalls waiting on ACKs	Minimal — tuned/auto-scaling window

Table 4.2 — Illustrative breakdown of where the 1ms -> 40ms increase comes from, and how tuning addresses each stage.

Cross-Case Comparative Analysis

Although each case study is analyzed independently, viewing them side by side reveals patterns that are useful for understanding network performance engineering as a discipline rather than four unrelated fixes. The table below summarizes the four cases along the same dimensions.

Dimension	Q1 — Video Conferencing	Q2 — DNS Redesign	Q3 — Distributed DNS	Q4 — Trading Latency
Layer primarily responsible	Transport + Application	Application (DNS)	Application (DNS)	Transport (TCP)
Trigger condition	Peak-hour congestion	Cross-continent distance	Promotional traffic burst	Market-open connection burst
Core diagnosis	UDP/RTP with weak buffering, or TCP misuse	Single-region authoritative server	Single server as latency + throughput ceiling	SYN queue, Nagle, flow control
Core fix category	Protocol choice + adaptive buffering	Anycast + pre-fetch + TTL tuning	Distributed, elastic Anycast mesh	Socket-level TCP configuration
Primary metric improved	Jitter, packet loss impact	Lookup time (~800ms -> <50ms)	Availability under burst load	Ack latency (40ms -> single-digit ms)

Table 5.1 — Side-by-side comparison of diagnosis and remedy across all four case studies.

Two broader patterns emerge from this comparison. First, three of the four cases (Q1, Q2, Q3) involve DNS or media delivery at the application layer, while only Q4 is a pure transport-layer configuration issue — reflecting how, in practice, application-visible performance problems are more often rooted in how a protocol is used and deployed than in the protocol's core specification being inadequate. Second, every recommended fix follows the same underlying strategy noted in the conclusion below: replace a fixed, centralized, or default-tuned behaviour with one that adapts to the actual traffic pattern of the workload in question.

Conclusion

Across all four cases, the recurring theme is that protocol behaviour which is correct and beneficial in one context (TCP's reliability, Nagle's coalescing, a single centralized server, a fixed connection backlog) becomes a performance liability when the traffic pattern or application requirement changes — real-time media, global user distribution, promotional traffic spikes, or market-open bursts. In each case, the diagnosis followed the same method: identify which layer and mechanism produces the observed symptom, understand why that mechanism exists and what trade-off it is making, and then recommend a targeted change (protocol choice, architectural redistribution, or configuration tuning) rather than a generic fix.

The four solutions also share a common design principle: move fixed, centralized, or static behaviour toward adaptive, distributed, and load-aware behaviour — adaptive jitter buffers instead of fixed ones, Anycast-distributed DNS instead of a single server, elastic regional capacity instead of fixed capacity, and burst-tuned TCP parameters instead of average-load defaults. This principle generalizes well beyond these four specific cases and reflects how production-scale network systems are engineered in practice.

References

- Internet Engineering Task Force (IETF), RFC 793 — Transmission Control Protocol.
- Internet Engineering Task Force (IETF), RFC 768 — User Datagram Protocol.
- Internet Engineering Task Force (IETF), RFC 3550 — RTP: A Transport Protocol for Real-Time Applications.
- Internet Engineering Task Force (IETF), RFC 1035 — Domain Names: Implementation and Specification.
- Internet Engineering Task Force (IETF), RFC 1546 — Host Anycasting Service.
- Internet Engineering Task Force (IETF), RFC 896 — Congestion Control in IP/TCP Internetworks (Nagle's Algorithm).
- Kurose, J. F., & Ross, K. W. — Computer Networking: A Top-Down Approach (course reference textbook).
- Stevens, W. R. — TCP/IP Illustrated, Volume 1: The Protocols.

Appendix — Assessment Rubric

Criteria	Weight	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of Case	25%	Thorough; all key issues identified	Good; most issues identified	Basic; some issues missed	Poor; problem not identified
Application of Concepts	25%	Effectively applies concepts in depth	Applies concepts with minor gaps	Limited application	Fails to apply concepts
Analysis	20%	In-depth, strong reasoning	Good, logical reasoning	Basic, limited depth	Weak or no analysis
Solution Design	20%	Innovative, feasible, well justified	Logical, adequately justified	Basic, weak justification	Incorrect / no solution
Clarity	10%	Clear, well-structured	Organized, understandable	Limited clarity	Poor clarity